

## APPENDIX

### A. System deployment architecture

The deployment architecture of **REDPARROT** is structured around a dual-path execution strategy (Short-chain and Long-chain) orchestrated by a DAG-based workflow engine. To manage the lifecycle of query skeletons, the system employs an offline-to-online pipeline: high-quality query templates are pre-constructed through K-means clustering and stored in a Milvus-backed vectorized repository, while the online serving layer utilizes a Query Erase module to extract structural patterns from raw NL inputs. The retrieval phase implements a multi-strategy hybrid mechanism, combining BM25, semantic embedding, and a weighted voting-based reranker to select the optimal dataset ID. For entity alignment, the architecture integrates a two-stage retrieval process that sequentially identifies dimension fields and their corresponding values to ensure semantic consistency.

All processing nodes, from intent parsing to DSL generation, are encapsulated within a modular microservice framework, enabling asynchronous performance monitoring and automatic fallback from short-chain template matching to the heterogeneous RAG-driven long-chain when similarity thresholds are not met. Additionally, experimental results incorporating the fallback mechanism are presented in Table II, demonstrating that both the ACC and P90 latency metrics consistently maintain superior performance.

TABLE I: Performance evaluation of the fallback mechanism from cache shortcut to full-generation workflow.

| Dataset   | Method                 | ACC   | P90   |
|-----------|------------------------|-------|-------|
| Commerce  | LONGCHAIN-DS           | 45.84 | 25.00 |
|           | REDPARROT w/o Fallback | 57.64 | 7.15  |
|           | REDPARROT w/ Fallback  | 48.55 | 20.89 |
| Community | LONGCHAIN-DS           | 60.67 | 34.00 |
|           | REDPARROT w/o Fallback | 59.02 | 7.74  |
|           | REDPARROT w/ Fallback  | 60.47 | 30.06 |
| Trading   | LONGCHAIN-DS           | 20.83 | 29.00 |
|           | REDPARROT w/o Fallback | 33.33 | 9.19  |
|           | REDPARROT w/ Fallback  | 23.08 | 25.43 |

### B. Cache updating strategy

We provide two strategies for updating the skeleton cache: a full rebuild and an incremental update. The baseline strategy triggers a comprehensive global rebuild whenever the volume of newly accumulated queries reaches a 10% threshold (approximately 580 instances relative to the initial pool of 5,854). While the full rebuild strategy updates the cache by re-processing the entire dataset, the resulting overhead becomes prohibitive at larger scales. To mitigate this, we introduce an online incremental update strategy that identifies representative candidates through connectivity-based filtering of incoming query clusters. A candidate is integrated into the cache only when its similarity  $s$  to existing templates satisfies specific threshold constraints. Specifically, a similarity of  $s > 0.95$

identifies high-confidence patterns for reinforcement, while  $s < 0.9$  indicates novel structural diversity that warrants inclusion. These high thresholds(0.9 and 0.95) are chosen because the similarity distribution between new and historical queries at Xiaohongshu is highly concentrated.

TABLE II: Comparison of different cache updating strategies.

| Dataset   | Strategy     | ACC (%) | Cov. (%) | Overhead (s)           |
|-----------|--------------|---------|----------|------------------------|
| Commerce  | N/A          | 46.55   | 22.13    | 50.11                  |
|           | Full Rebuild | 41.34   | 22.13    | 48.7                   |
|           | Incremental  | 51.72   | 22.13    | 9.48 ( $\times 5.13$ ) |
| Community | N/A          | 64.28   | 12.17    | 15.66                  |
|           | Full Rebuild | 58.26   | 14.71    | 16.61                  |
|           | Incremental  | 60.0    | 13.04    | 4.93 ( $\times 3.36$ ) |
| Trading   | N/A          | 76.19   | 17.79    | 14.38                  |
|           | Full Rebuild | 80.02   | 17.79    | 19.62                  |
|           | Incremental  | 76.19   | 17.79    | 7.47 ( $\times 2.62$ ) |

Experimental results (Table II) reveal that while accuracy fluctuates across datasets due to the high similarity between incremental and historical queries, the incremental update strategy delivers a transformative reduction in computational latency. Achieving an average  $3.7\times$  speedup, this approach effectively sidesteps the prohibitive overhead of global re-computation, particularly in the scale-heavy **RED-commerce** domain where it realizes a  $5.13\times$  efficiency gain. Although the full rebuild occasionally underperforms—likely because exhaustive re-processing introduces global noise—the incremental strategy maintains competitive precision and superior structural coverage. The incremental strategy effectively preserves the precision of the DSL pool while ensuring the feasibility of continuous, low-overhead cache evolution.

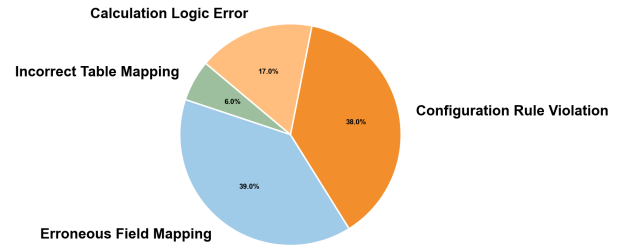


Fig. 1: Distribution of execution error categories in NL-to-DSL generation.

### C. Skeleton validation using LLM-as-a-Judge

To ensure the high fidelity of the skeleton cache, we implement a discriminative LLM-as-a-Judge framework designed to identify extraction errors arising from non-isomorphic semantic mappings between natural language entities and DSL values. Utilizing a Chain-of-Thought (CoT) reasoning process, the judge evaluates each candidate across three dimensions: Semantic Completeness, Entity Grounding, and Structural Logic. The specific prompt template for this validation process is provided below.

## Prompt for skeleton validation using LLM-as-a-Judge

**Role:** You are an expert Data Auditor specializing in Business Intelligence (BI) and Semantic Parsing. Your task is to verify if a generated Domain-Specific Language (DSL) snippet accurately represents the user’s Natural Language Query (NLQ).

### Component Definitions:

- 1) **Measure:** Quantitative values computed via aggregation (e.g., SUM, COUNT) or arithmetic operations.
- 2) **Dimension:** Categorical fields used to group, segment, or disaggregate data.
- 3) **Filter:** Conditions constraining data (e.g., WHERE/HAVING clauses).

**Evaluation Logic (Entity Grounding):** You must pay critical attention to **Entity Alignment**. Often, NLQ entities (e.g., "last year") map to specific DSL values (e.g., "2025"). You must determine if these mappings are logically sound or if the LLM has "hallucinated" entities not present or implied by the query.

### Execution Steps:

- 1) **Extraction:** Identify all Measures, Dimensions, and Filters requested in the NLQ.
- 2) **Comparison:** Map the extracted NLQ requirements to the provided DSL components.
- 3) **Conflict Detection:** Check for **Entity Mismatch** (DSL values that contradict the query), **Logic Drift** (Measures mapped as Dimensions, or missing Filters), and **Over-generation** (Adding constraints not mentioned in the query).

### Input Data:

- **User Query:** {{user\_query}}
- **Generated DSL:** {{generated\_dsl}}

**Response Format:** Please provide your audit in the following structure: 1) **Entity Mapping Table** (including Corresponding DSL Value and Status: Match/Mismatch/Hallucination); 2) **Component Verification** (Measure, Dimension, and Filter Check with Pass/Fail reasons); 3) **Final Alignment Score (0-10)**; 4) **Conclusion** ([VALID] or [INVALID]).

Quantitative evaluation on Xiaohongshu production datasets demonstrates that this mechanism achieves approximately 98% extraction accuracy, successfully filtering inconsistent mappings before cache commitment. By serving as a rigorous secondary safeguard, this framework ensures that only high-quality skeletons are utilized as shortcuts, while ambiguous candidates trigger a cache miss to engage the full-generation fallback, thereby preserving end-to-end precision.

### D. Error analysis

In this section, we examine cases where the generated DSL failed to accurately retrieve the intended data or satisfy the complex analytical constraints of the enterprise environment. We categorized these errors into four primary groups: (1) Wrong Table Mapping, (2) Wrong Field Mapping, (3) Configuration Rule Violation, and (4) Calculation Logic Error. Fig. 1 illustrates the distribution of each category among the sampled failure cases. In the following sections, we will discuss each of these categories in more detail.

1) *Wrong Table Mapping:* There is a small probability of table selection errors caused by the cache hitting an inaccurate skeleton. As shown in Fig. 2, the ground truth is "Core\_Note\_Daily\_Incremental\_Wide\_Table", while the actual output is "Note\_Metric\_Table".

2) *Wrong Field Mapping:* Ambiguous queries or erroneous field retrieval from the RAG module can lead to inaccuracies

in the final field values. As illustrated in Fig. 3, the Measure field is missing the "Operational\_Spend" field.

3) *Configuration Rule Violation:* During the DSL rewriting process, configuration rules for Dimensions, Measures, and Filters, as well as specific business field logic, are provided. However, errors may arise when encountering atypical fields not covered by these predefined rules. As illustrated in Fig. 4, while the "BrandTag" in the original question is expected to be in Chinese (??? denotes a brand name masked for privacy), the actual retrieved field value is incorrectly mapped to its English counterpart.

4) *Calculation Logic Error:* For computationally complex queries, it is difficult for models to accurately complete tasks based solely on a set of predefined calculation functions. As shown in Fig. 5, when filtering the engagement and CTR of various note types for specific days of the week within a certain date range, both the specific dates and the days of the week should belong to the filter conditions of the "Note\_Posting\_Day" field, rather than separately "filtering Date" and "Note\_Posting\_Day".

**Question:**

Retrieve the word cloud performance for notes containing the keyword '???' over the last 30 days.

**DSL:**

```
Table: Core_Note_Daily_Incremental_Wide_Table
Dimension: []
Measure: [{"field": "Community_Impressions", "aggregation": "sum"}, {"field": "Community_Clicks", "aggregation": "sum"}, {"field": "Community_Engagement", "aggregation": "sum"}, {"field": "Community_Collects", "aggregation": "sum"}, {"field": "Community_Comments", "aggregation": "sum"}, {"field": "Community_Follows", "aggregation": "sum"}]
Filter: [{"field": "Keyword", "type": "Contains", "condition": "???"}, {"field": "Note_Posting_Day", "type": "Relative_Time", "condition": "Last_30_Days"}, {"field": "Is_Valid_Note", "type": "Equals", "condition": 1}]
```

**Ground Truth:**

```
Table: Core_Note_Daily_Incremental_Wide_Table
Dimension: []
Measure: [{"field": "Community_Impressions", "aggregation": "sum"}, {"field": "Community_Clicks", "aggregation": "sum"}, {"field": "Community_Engagement", "aggregation": "sum"}, {"field": "Community_Collects", "aggregation": "sum"}, {"field": "Community_Comments", "aggregation": "sum"}, {"field": "Community_Follows", "aggregation": "sum"}]
Filter: [{"field": "L4_Business_Category", "type": "Exact_Match", "condition": "Teacher_Certification"}, {"field": "Note_Posting_Day", "type": "Relative_Time", "condition": "Last_30_Days"}, {"field": "Is_Valid_Note", "type": "Equals", "condition": 1}]
```

Fig. 2: An Example of Wrong Table Mapping.

**Question:**

Identify the high-delivery notes associated with the BrandTag '???' during the period from January to December 2025.

**DSL:**

```
Table: Commercial_Note_Metric_Wide_Table
Dimension: [{"field": "Note_ID"}, {"field": "Note_Title"}, {"field": "Note_Link"}]
Measure: [{"field": "Ad_Impressions", "aggregation": "Aggregated"}, {"field": "Ad_Clicks", "aggregation": "Aggregated"}, {"field": "Seeding_Ad_Turnover", "aggregation": "Aggregated"}, {"field": "Bidding_Ad_Turnover", "aggregation": "Aggregated"}]
Filter: [{"field": "Brand_Tag", "type": "Fuzzy_Match", "condition": "???"}, {"field": "Date", "type": "Absolute_Time", "condition": ["2025-01-01", "2025-12-31"]}]
```

**Ground Truth:**

```
Table: Commercial_Note_Metric_Wide_Table
Dimension: [{"field": "Note_ID"}, {"field": "Note_Title"}, {"field": "Note_Link"}]
Measure: [{"field": "Ad_Impressions", "aggregation": "Aggregated"}, {"field": "Ad_Clicks", "aggregation": "Aggregated"}, {"field": "Seeding_Ad_Turnover", "aggregation": "Aggregated"}, {"field": "Bidding_Ad_Turnover", "aggregation": "Aggregated"}, {"field": "Operational_Spend", "aggregation": "Aggregated"}]
Filter: [{"field": "Brand_Tag", "type": "Fuzzy_Match", "condition": "???"}, {"field": "Date", "type": "Absolute_Time", "condition": ["2025-01-01", "2025-12-31"]}]
```

Fig. 3: An Example of Wrong Field Mapping.

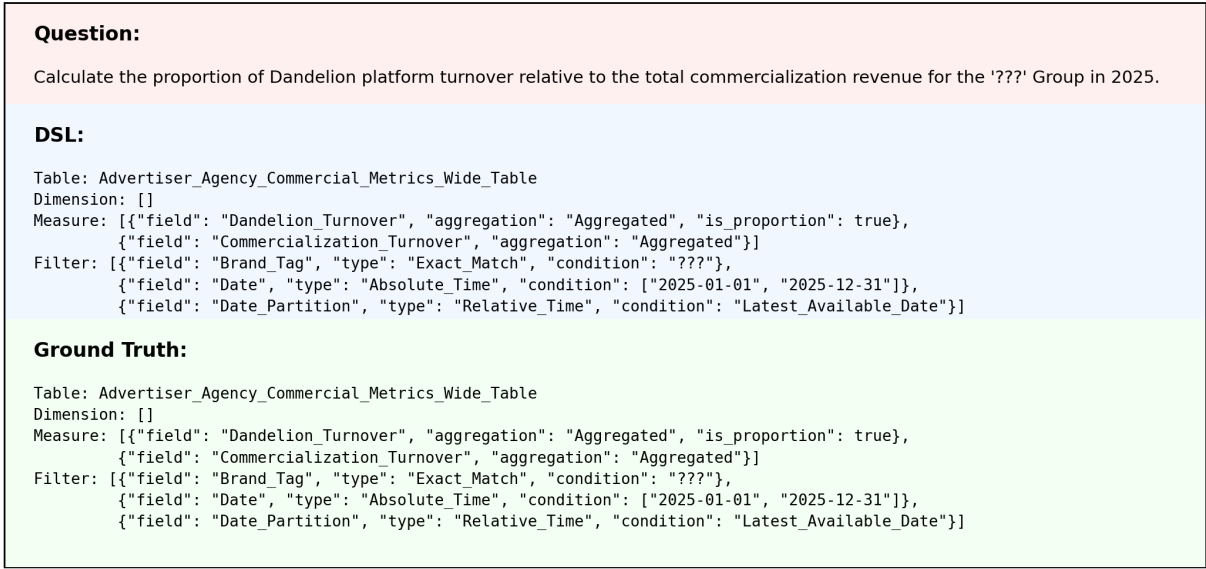


Fig. 4: An Example of Configuration Rule Violation.

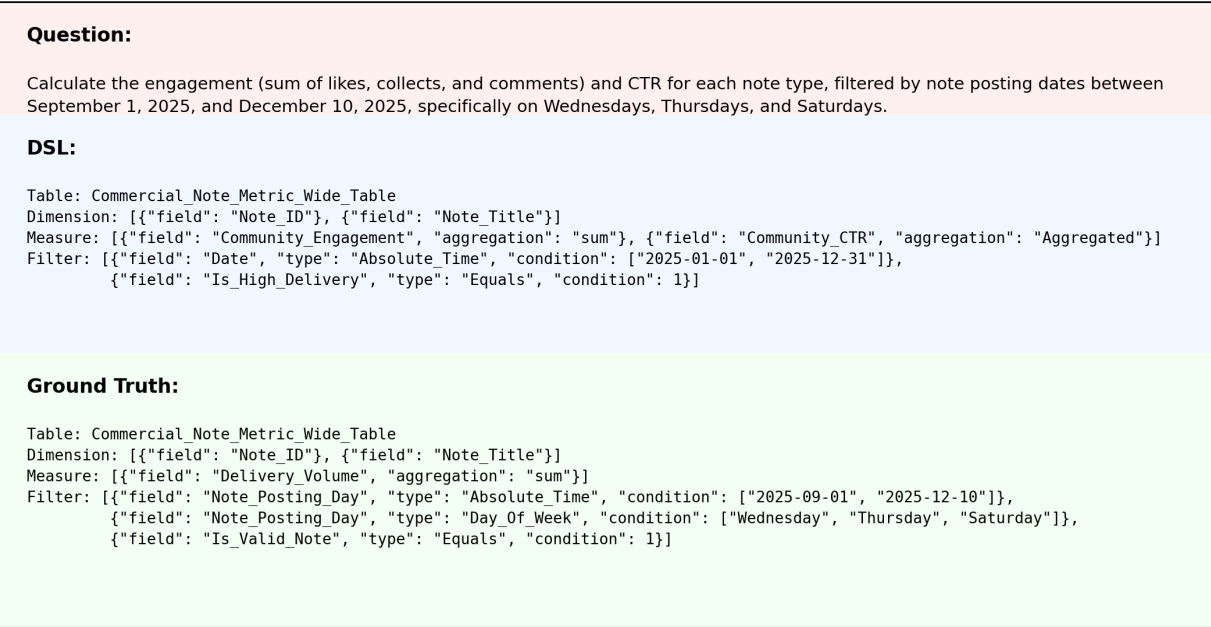


Fig. 5: An Example of Calculation Logic Error.